

32-BIT ASSEMBLY CHEAT SHEET

REGISTER MAP

REG	ARCHITECTURE / ABI
r0 / zero	hardwired zero
r1–r4	arguments; caller-saved
r5–r12	general; callee-saved
r13 / sp	banked stack pointer
r14 / lr	link / return register
r15	caller-saved temporary

Special state

PC	program counter
SR	status / privilege control

ABI quick facts

args: r1, r2, r3, r4; later args on stack
return: r1 (32-bit), r1:r2 (64-bit, low word first)
stack grows downward; 16-byte aligned at calls
no red zone; nested calls preserve incoming lr

SR BITS

CF 0	OF 1	IPL 2–4	S 5	reserved 6–31
------	------	---------	-----	---------------

CF: carry out; subtraction sets CF on unsigned borrow.
OF: signed two's-complement overflow.
ADC/SBC consume old CF as carry/borrow input.
No N or Z flags: comparisons write canonical 0 or 1.

MEMORY MODEL

32-bit words • little-endian bytes • natural alignment
byte: any address halfword: address % 2 == 0
word: address % 4 == 0 effective = base + signed disp13

lbu r1, 1(r5) lh r2, 2(r5) lw r3, 4(r5)
sb r3, 0(r5) sh r3, 2(r5) sw r3, 4(r5)

INSTRUCTION QUICK REFERENCE

All R-format operations: rd, ra, rb

ARITHMETIC	OPERATION
add/adc rd, ra, rb	ra + rb / + old CF
sub/sbc rd, ra, rb	ra – rb / – old CF
mul/mulh/mulhu	rd, ra, rb; low / signed high / unsigned high
div/divu rd, ra, rb	signed / unsigned quotient
rem/remu rd, ra, rb	signed / unsigned remainder
and/or/xor rd, ra, rb	bitwise
not rd, ra, r0	bitwise complement; rb must be r0
sll/srl/sra rd, ra, rb	logical left / right / arithmetic right
rol/ror rd, ra, rb	rotate left / right
seq/sne rd, ra, rb	equal / not equal → 0 or 1
slt/sge rd, ra, rb	signed < / ≥ → 0 or 1
sltu/sgeu rd, ra, rb	unsigned < / ≥ → 0 or 1

IMMEDIATE rd, ra, imm

addi / subi signed 12-bit arithmetic; flags update
andi / ori / xori zero-extended 12-bit mask
slli / srli / srai immediate shift count 0–31

MEMORY rt, displacement(base)

lb / lbu signed / zero-extended byte
lh / lhu signed / zero-extended halfword
lw 32-bit word sb / sh / sw stores

BRANCH ra, rb, target

beq bne blt bge bltu bgeu
signed: blt/bge unsigned: bltu/bgeu
target = (PC + 4) + sign_extend(disp << 2)

CONTROL / SYSTEM

j target jal target jr ra jalr ra
mfsr rd, sysreg mtsr sysreg, rs rfe
syscall 0 tlbflush [rN] cas rd, rs, 0(ra)
break halt

CONTROL TRANSFER

beq r1, r2, equal
jal function # lr = PC + 4
jr lr # return
jalr r5 # indirect call
j loop # no link

$$\text{target} = (\text{PC} + 4) + \text{sign_extend}(\text{disp} \times 4)$$

Branches and direct jumps use PC+4 as the base.
Displacements are signed instruction units (×4).
Targets must be 4-byte aligned.

INSTRUCTION FORMATS

FORMAT	FIELDS
R	op rd ra rb fn
I	op rd ra sub imm12
MEM	op rt base size disp13
B	op cond ra rb disp13
J	op disp26
IND	op link ra
U	op rd imm20
SYS	op sysop reg sysreg
CAS	op rd rs ra

ATOMIC CAS

retry:
lw r1, 0(r5) # expected
add r3, r1, r0
addi r2, r1, 1 # desired
cas r3, r2, 0(r5) # r3 = old
bne r3, r1, retry

CAS compares original rd, stores rs on match,
then always writes observed old value to rd.

ASSEMBLY PROGRAMMING QUICK REFERENCE

ABI AND STACK FRAME

REGISTERS	ROLE
r1–r4	args; caller-saved
r5–r12	callee-saved
r13 / sp	downward stack; callee-managed
r14 / lr	return link; save before nested call
r15	caller-saved temporary

Nested-call frame (stack grows ↓)

higher addresses
saved r5 / locals
saved lr
sp →

```
caller:
addi sp, sp, -16
sw r5, 0(sp)
sw lr, 4(sp)
jal callee
lw lr, 4(sp)
lw r5, 0(sp)
addi sp, sp, 16
ret
```

Call boundaries are 16-byte aligned; no red zone.
r1 returns 32-bit values; r1:r2 returns 64-bit values.

GAS SYNTAX AND PATTERNS

```
.section .text .section .rodata
.section .data .section .bss
.global main .type main,@function
.size main, .-main
.byte 0x7f .short 0x1234
.word 0x12345678 .space 64
.ascii "REM\n" .asciz "REM"
.align 2 label:
l: j lb # numeric local label
# comment .equ N, 10
```

Full symbol address (HI20 / LO12)

```
lui r5, %hi(symbol) # R_MYEMU_HI20
ori r5, r5, %lo(symbol) # R_MYEMU_LO12
```

Supported pseudo-instructions

```
li rd, constant # addi or lui/ori pair
call symbol # jal symbol
ret # jr r14
```

Common patterns

```
li r1, 0x12345678
lw r2, 0(r5) # read
sw r2, 4(r5) # write
loop: addi r1, r1, -1
bne r1, r0, loop
add r1, r1, r3 # 64-bit low
adc r2, r2, r4 # 64-bit high
sub r1, r1, r3 # 64-bit low
sbc r2, r2, r4 # 64-bit high
```

LINUX SYSCALL ABI

```
r1 = syscall number
r2–r7 = six arguments
syscall 0
r1 = signed return value
r1 < 0 → -errno
```

NR	CALL	PURPOSE
56	openat	open path
57	close	close fd
61	getdents64	read directory
62	lseek	move offset
63	read	read bytes
64	write	write bytes
93/94	exit/group	terminate
160	uname	kernel identity
214/215	brk/munmap	memory
222	mmap	map memory
403	clock_gettime	time

Write a string

```
.section .rodata
msg: .ascii "Hello from REM\n"
msg_end:
.section .text
.global _start
_start:
lui r3, %hi(msg)
ori r3, r3, %lo(msg)
li r1, 64 # write
li r2, 1 # stdout
li r4, msg_end-msg
syscall 0
li r1, 94 # exit_group
li r2, 0
syscall 0
```

Build / inspect / run

```
myemulator2-elf-as -o x.o x.S
myemulator2-elf-ld -T toolchain/userspace/myemulator2-user.ld -o x.x.o
myemulator2-elf-objdump -dr x
myemulator2-elf-readelf -h -s -r x
qemu-system-myemulator32 -M myemulator32 \
-kernel vmlinux -nographic
```